

RustPower OPF 算法设计文档

面向读者：熟悉电力系统基础、希望理解代码内部如何工作的研究者/工程师。

测试案例：IEEE118（118母线）；所有性能数据均来自

```
cargo run --release --features klu --example v5_compare。
```

目录

1. 问题公式化（OPF）
2. 求解器：内点法 PIPS
3. 迭代循环内的五个计算阶段
4. 代码层次结构
5. KKT 矩阵：从 V4 到 V5.6 的演化
6. 关键数据结构
7. 实测性能对比
8. 各版本改进一览

1. 问题公式化（OPF）

1.1 决策变量

采用 MATPOWER / PYPOWER 的标准变量排列：

$$x = \begin{bmatrix} V_a(nb) & | & V_m(nb) & | & P_g(ng) & | & Q_g(ng) \end{bmatrix}$$

电压相角 电压幅值 有功出力 无功出力

其中 nb = 母线数， ng = 发电机数， $nx = 2 \cdot nb + 2 \cdot ng$ 。

1.2 目标函数

多项式发电成本（以欧元 / MW 为单位）：

$$f(x) = \sum_{g=1}^{n_g} [c_{2,g} \cdot (P_g \cdot S_{base})^2 + c_{1,g} \cdot P_g \cdot S_{base} + c_{0,g}]$$

代码位置： `src/opf/cost.rs`

1.3 等式约束 $g(x) = 0$ （功率平衡）

$$g(x) = \begin{bmatrix} \text{Re}(V \odot \overline{Y_{bus} V}) - C_g P_g + P_d \\ \text{Im}(V \odot \overline{Y_{bus} V}) - C_g Q_g + Q_d \end{bmatrix} \in \mathbb{R}^{2n_b}$$

即每条母线的 P / Q 注入平衡误差（功率潮流方程残差）。

1.4 不等式约束 $h(x) \leq 0$ （支路潮流限制）

$$h_l = |S_{f,l}|^2 - S_{max,l}^2 \leq 0, \quad h_{n_l+l} = |S_{t,l}|^2 - S_{max,l}^2 \leq 0$$

每条支路两端的视在功率平方限制（共 $2 \cdot n_l$ 个不等式）。

代码位置： `src/opf/constraints.rs`

1.5 变量界

- 电压幅值上下界： $V_{min} \leq V_m \leq V_{max}$
- 有功/无功出力界： $P_{g,min} \leq P_g \leq P_{g,max}$, $Q_{g,min} \leq Q_g \leq Q_{g,max}$
- 参考母线相角固定： $V_{a,ref} = 0$ （等式约束 $x_{min} == x_{max}$ ）

2. 求解器：内点法 PIPS

PIPS（Predictor-Corrector Interior-Point Solver）是对原始-对偶内点法的一种实现，其标准形式为：

$$\min f(x) \quad \text{s.t.} \quad g(x) = 0, \quad h(x) \leq 0, \quad x_{min} \leq x \leq x_{max}$$

引入松弛变量 z （ $h(x) + z = 0, z > 0$ ）和对偶变量 λ （等式）、 μ （不等式），形成扩展拉格朗日：

$$L = f + \lambda^T g + \mu^T h$$

收敛判据（均需同时满足）：

条件	含义	阈值
feascond	约束可行性 (g、h 残差)	1e-6
gradcond	梯度条件 (KKT 梯度)	1e-6
compcond	互补松弛 $z \cdot \mu$	1e-6
costcond	目标函数变化率	1e-6

代码位置： `src/opf/pips.rs`

3. 迭代循环内的五个计算阶段

每次 PIPS 迭代均分为五个有独立计时的阶段：

开始迭代 i

① Hess

拉格朗日 Hessian 矩阵 L_{xx}
(或融合KKT直接填值)

② G/H

约束值 g, h 和
Jacobian dg, dh

③ KKT

组装 $(nx+neq) \times (nx+neq)$
鞍点矩阵 $K = [M, dg; dg^T, 0]$

④ Solve

KLU 稀疏直接求解
 $\Delta x, \Delta \lambda, \Delta z, \Delta \mu$

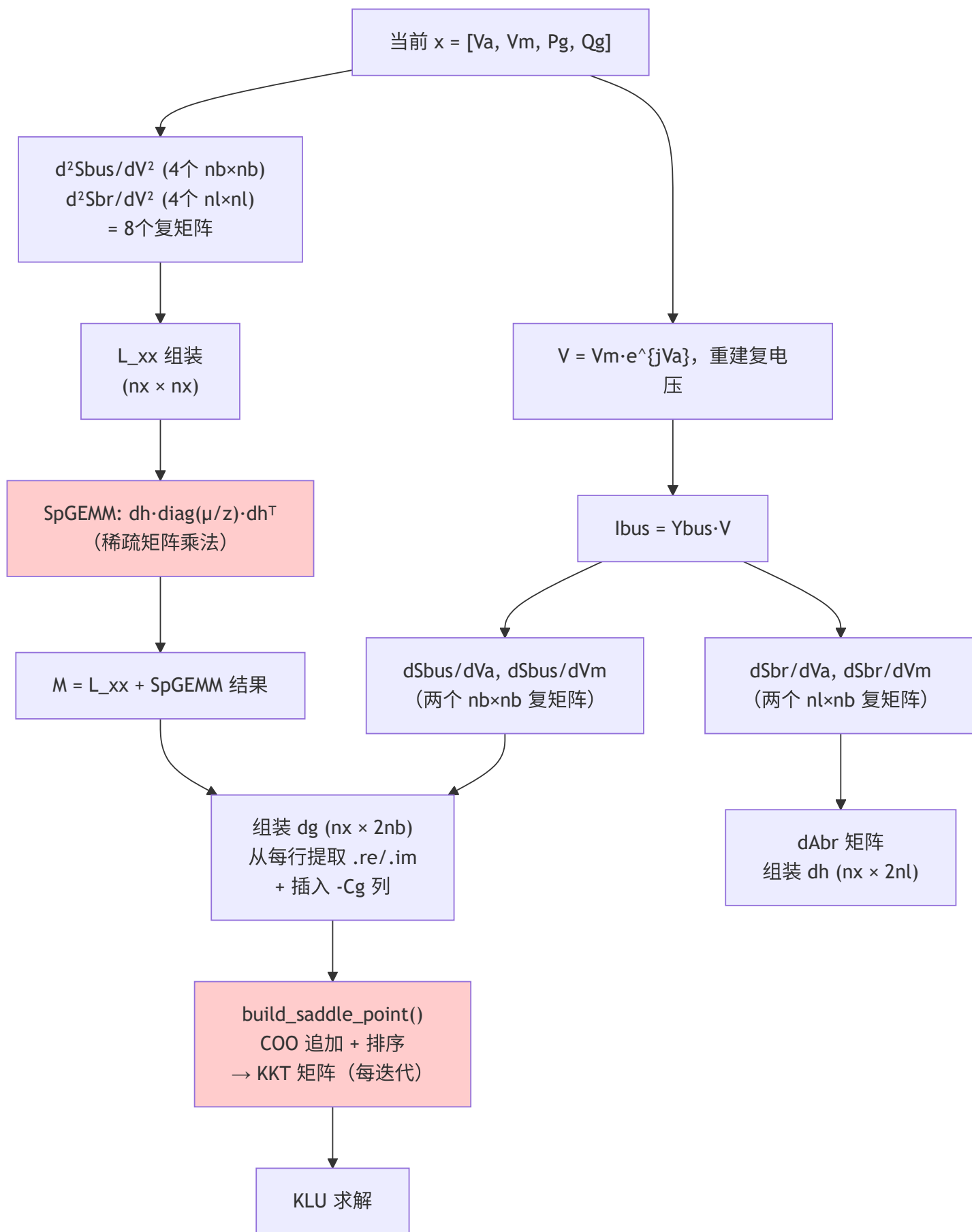
⑤ 步长 + 更新

计算 α_p, α_d 保持 $z > 0, \mu > 0$
 $x += \alpha \cdot \Delta x, \lambda += \alpha \cdot \Delta \lambda, \dots$

收敛?

否

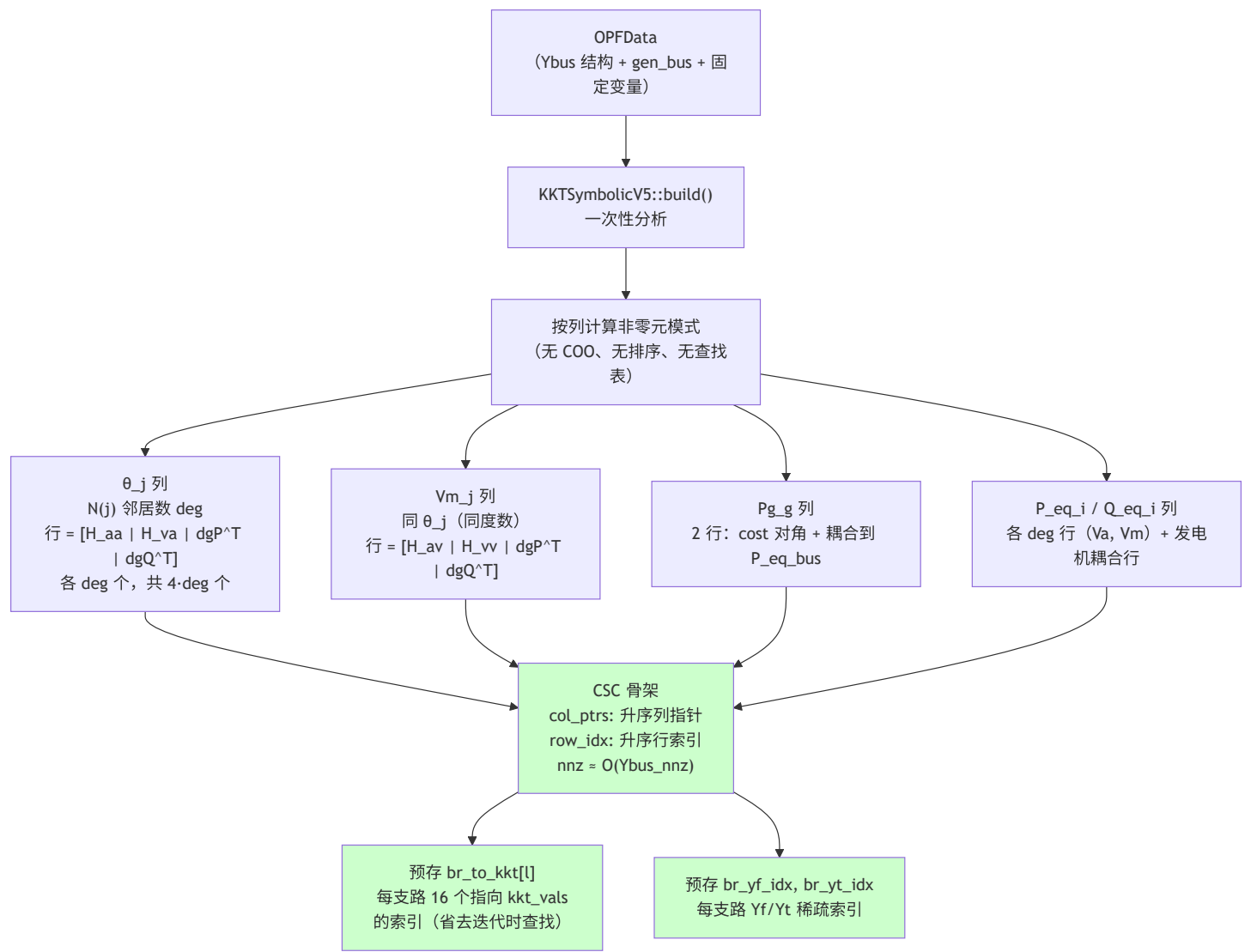
5.1 V4: 基线路径（每迭代重建所有矩阵）



V4 瓶颈：每迭代做一次 $\text{SpGEMM}(\text{dh} \cdot \text{diag} \cdot \text{dh}^T) + \text{build_saddle_point}(\text{COO} \rightarrow \text{CSC} + \text{排序})$ ，合计 **4.2 ms / 14 迭代**。

5.2 V5 符号骨架（一次性构建）

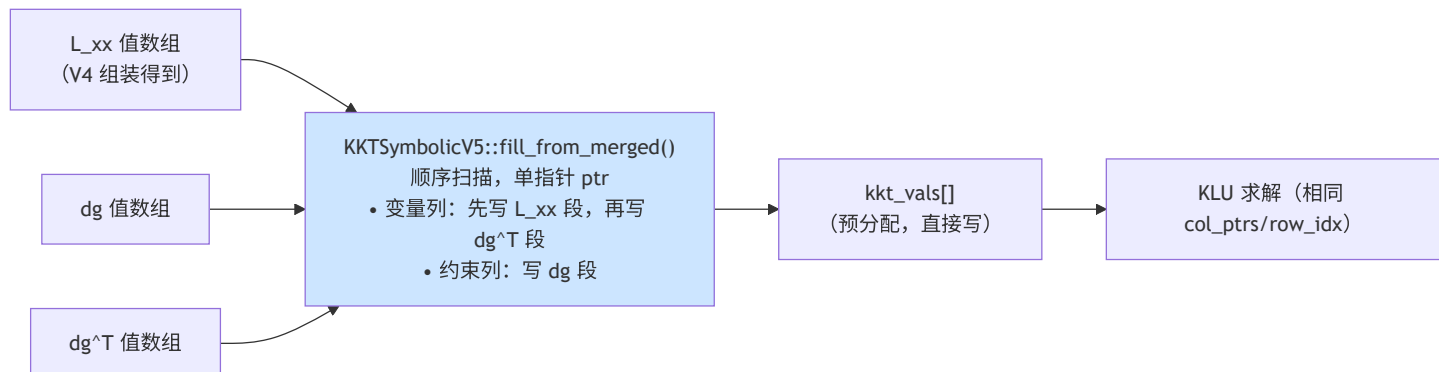
核心洞察：KKT 的非零元位置（稀疏结构）由 Ybus 拓扑决定，与 x 、 λ 、 μ 无关。因此只需在求解开始前构建一次 CSC 骨架（`col_ptrs`，`row_idx`），迭代中只更新 `values` 数组。



代码位置： `src/new_opf/v5_kkt.rs`

5.3 V5.0：流式数值填充

有了符号骨架后，V5.0 仍沿用 V4 的 L_{xx} 和 dg 矩阵，但用一个**单一前向指针流**替换 `build_saddle_point`：



V5.0 消除了：COO 追加、排序、`build_saddle_point` 中的 COO→CSC 转换。
KKT 阶段耗时：**4.20 ms → 1.16 ms (3.6×)**。

5.4 V5.2：块算子融合（无中间矩阵）

V5.2 彻底消除 L_{xx} 、 dg 、 dg^T 矩阵。所有 Hessian / Jacobian 值**直接计算并写入** `kkt_vals`。

$x = [V_a, V_m, P_g, Q_g]$

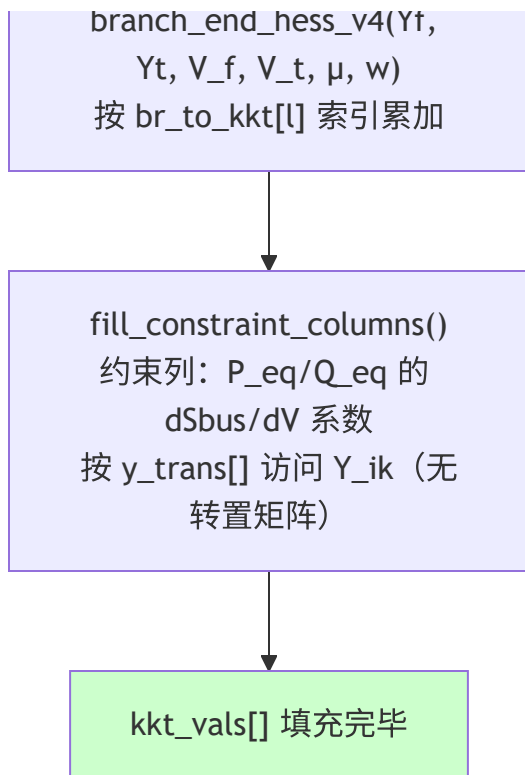
预计算全局向量 $O(nb)$
 $V, l_{bus}, lam_v, d_{lam},$
 inv_{vmag}

for k in $0..nb$ (Ybus 列遍历)
ColCtx { $vk, vnorm_k,$
 $lam_v_k, ibus_k, d_{lam_k}$ }

for i in $N(k)$ (邻居遍历)
NbrCtx { $vi, y_{ik}, y_{ki},$
 lam_v_i, is_{diag} }

8 个块算子 (纯函数
 $ColCtx \times NbrCtx \rightarrow f64$)
 $H_{aa}, H_{va} \rightarrow$
 $kkt_vals[th0 + 0 \cdot deg + off]$
 $H_{av}, H_{vv} \rightarrow$
 $kkt_vals[vm0 + 0 \cdot deg + off]$
 $dP/d\theta, dQ/d\theta \rightarrow$
 $kkt_vals[th0 + 2 \cdot deg + off]$
 $dP/dV_m, dQ/dV_m \rightarrow$
 $kkt_vals[vm0 + 2 \cdot deg + off]$

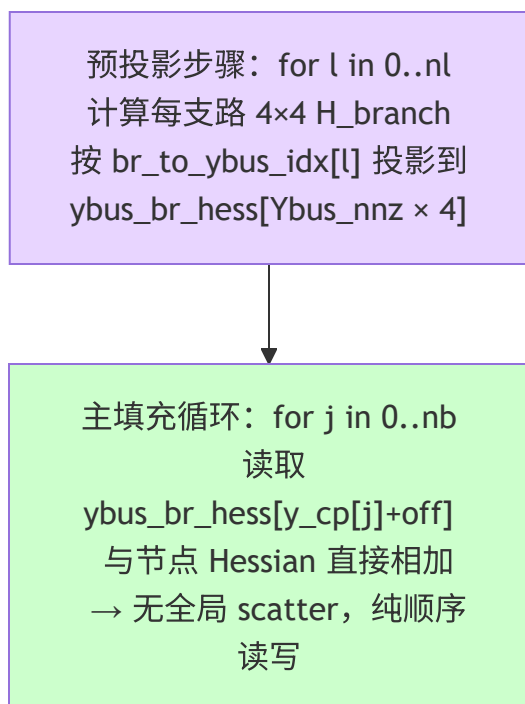
for l in $0..nl$ (支路
Hessian)



V5.2 消除了：所有中间矩阵（无 1_{xx} 、无 dg 、无 dg^T 、无 SpGEMM）。
KKT 阶段耗时：**1.16 ms $\rightarrow$ 174 μ s (6.7 $\times$)。**

5.5 V5.3：支路 Hessian 预投影（分区同构）

V5.3 在 V5.2 基础上进一步优化支路 Hessian 的内存访问模式：



原理：支路 Hessian 与 Ybus 具有同构拓扑结构——每条支路的 4 个 (f,t) 节点对正好对应 Ybus 中的 4 个非零元位置。预投影后主循环变为纯顺序访问，对大规模（pegase9241 级别）缓存友好性显著提升。

5.6 V5.5 / V5.6: 零分配 Jacobian + 静态 G/H 结构

V5.5: V55Evaluator (一次性初始化)

分析 dgn/dhn 的 CSC 结构
构建 Ybus-nnz $\rightarrow$ dgn-nnz
索引映射

va_to_dgn_p/q[ybus_nnz]
vm_to_dgn_p/q[ybus_nnz]

daf_va/vm_to_dhn[yf_nnz]

V5.5 迭代内 update()

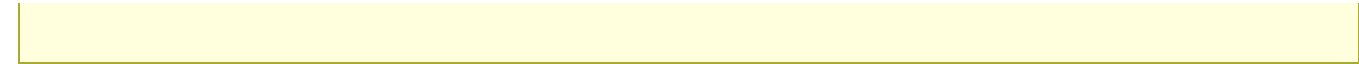
Ybus·V, Yf·V, Yt·V
(必要的矩阵向量积)

```
for k in 0..nb, for i in N(k)
  dsa = j·V_i·(lbus_i -
    Y_ik·V_i)*
  dsm =
  V_i·(Y_ik·Vnorm_k)* + ...
  dgn_v[va_to_dgn_p[idx]]
    = dsa.re
  dgn_v[va_to_dgn_q[idx]]
    = dsa.im
  ... (直接写入预分配数组)
```

V5.6: V56Evaluator (额外消除线性约束开销)

初始化时构建
dg = [dgn | ae^T] (静态结构)
dh = [dhn | ai^T] (静态结构)
ae^T/ai^T 后缀值永久固定 (±1)

迭代内只做:
V5.5.update() $\rightarrow$ 更新
dgn/dhn 前缀值
g[2nb:] = ae·x - be (稠密数组)
h[2nl:] = ai·x - bi
无 transpose, 无 hstack,
无 merge



V5.5 消除了： dSbus_dVa 、 dSbus_dVm 、 dAbr 等中间矩阵的构造。

V5.6 额外消除了： merge_constraints 、 transpose(ai) 、 hstack_csc 的每迭代调用。

G/H 阶段耗时：**3.21 ms → 477 μs (6.7×)**。

6. 关键数据结构

6.1 OPFData（问题数据， src/opf/problem.rs ）

字段	类型	含义
ybus	CscMatrix<Complex64>	节点导纳矩阵 (nb×nb)
yf , yt	CscMatrix<Complex64>	支路首/末端导纳 (nl×nb)
f_buses , t_buses	Vec<usize>	支路首/末端母线索引
cg	CscMatrix<f64>	发电机-母线关联矩阵 (nb×ng)
cost_coeffs	Vec<[f64;3]>	每台发电机 [c2, c1, c0]
rate_a	Vec<f64>	支路视在功率限制 [p.u.]

6.2 KKTSymbolicV5（符号骨架， src/new_opf/v5_kkt.rs ）

字段	类型	含义
col_ptrs	Vec<usize>	KKT CSC 列指针（维度 dim+1）
row_idx	Vec<usize>	KKT CSC 行索引（维度 nnz）
br_to_kkt	Vec<[usize;16]>	每支路 → kkt_vals 中 4×4 块的 16 个索引
br_yf_idx	Vec<[usize;2]>	预存 Yf 每行的稀疏索引
gens_at_bus	Vec<Vec<usize>>	每母线上的发电机列表
ieq	Vec<usize>	固定变量索引（Va_ref 等）

6.3 各版本求解器入口对照

调用函数	路径	内核组合
<code>pips()</code>	<code>new_opf/pips.rs</code>	V4 rect fill + <code>build_saddle_point</code>
<code>pips_v5()</code>	同上	V4 rect fill + V5.0 流式 fill
<code>pips_v5_2()</code>	同上	V5.2 块算子 (<code>fused_assembly</code>)
<code>pips_v5_3()</code>	同上	V5.3 分区同构 (<code>fused_assembly</code>)
<code>pips_v5_5()</code>	同上	V5.3 KKT + V5.5 零分配 Jacobian
<code>pips_v5_6()</code>	同上	V5.3 KKT + V5.6 静态 G/H

7. 实测性能对比

测试条件：IEEE118，warm start，14 PIPS 迭代，KLU 求解器，`--release`。
数值均为 14 次迭代的总耗时。

版本	Hess	G/H	KKT	Solv (KLU)	组装合计 ¹	端到端
V4 （基线）	660 μ s	3210 μ s	4200 μ s	4120 μ s	8070 μs	12190 μs
V5.0 流式 KKT	565 μ s	3580 μ s	1160 μ s	4460 μ s	5305 μ s	9765 μ s
V5.2 块算子	655 μ s	2535 μ s	174 μ s	3503 μ s	3364 μ s	6867 μ s
V5.3 分区同构	778 μ s	3004 μ s	157 μ s	3695 μ s	3939 μ s	7634 μ s
V5.5 零分配 J	687 μ s	817 μ s	153 μ s	3472 μ s	1657 μs	5129 μ s
V5.6 静态G/H	696 μ s	477 μ s	145 μ s	3458 μ s	1318 μs	4778 μs

¹ 组装合计 = Hess + G/H + KKT（RustPower 控制的部分；Solv 为外部 KLU）

各阶段加速比（V4 → V5.6）：

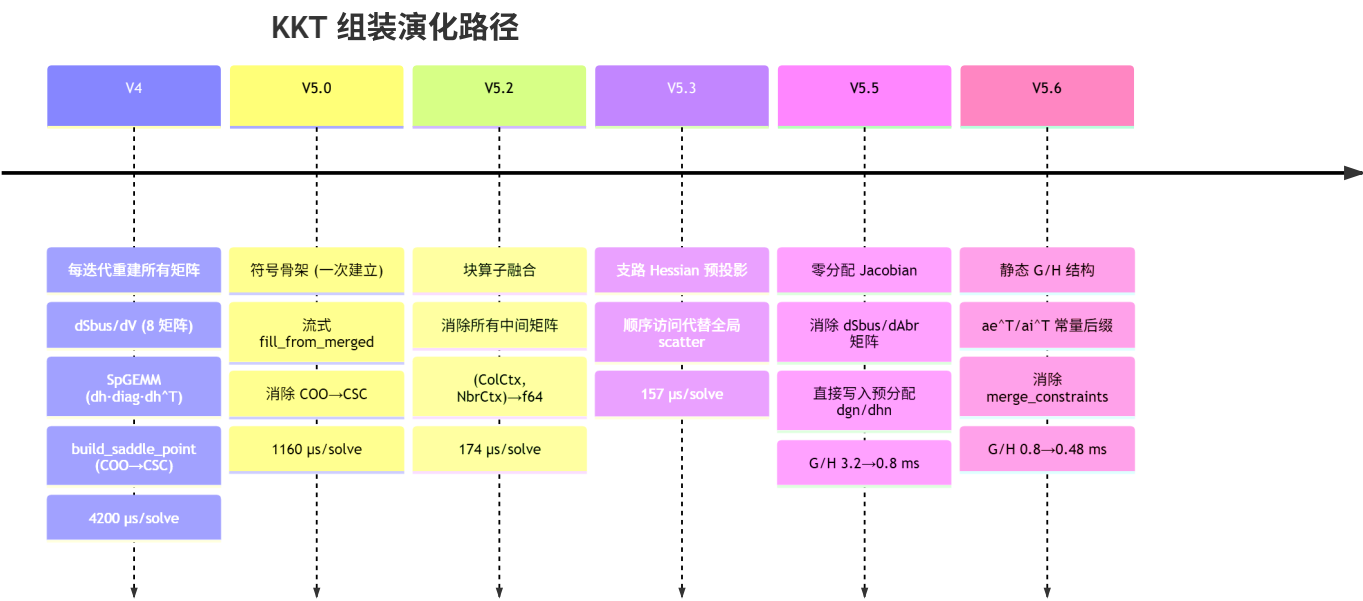
KKT 组装: $4200 \rightarrow 145 \mu s = 29\times$ (消除 SpGEMM + COO→CSC)
 G/H 计算: $3210 \rightarrow 477 \mu s = 6.7\times$ (消除中间矩阵 + 静态结构)
 Hess: $660 \rightarrow 696 \mu s \approx$ 持平 (已是 V4 瓶颈以外)
 Solv: $4120 \rightarrow 3458 \mu s \approx$ 持平 (持久化 KLU, 由外部库决定)

组装合计: $8070 \rightarrow 1318 \mu s = 6.1\times$
 端到端: $12190 \rightarrow 4778 \mu s = 2.55\times$

当前瓶颈

V5.6 的 solv 占端到端总时间的 **72%** ($3458/4778$), 而 V4 时仅占 **34%**。意味着进一步的 OPF 性能提升需要针对线性求解器 (KLU 并行化、超节点重排序), 而非 KKT 组装。

8. 各版本改进一览



关键设计原则总结

- 分离符号与数值**: KKT 的稀疏结构 (由 Ybus 拓扑决定) 只分析一次, 数值每迭代覆写。
- 流式写入**: 所有数值按 CSC 列顺序单向写入, 避免随机访问和 scatter。
- 消除分配**: 所有性能关键路径的 vec 只分配一次 (初始化时), 迭代中只改写 &mut [f64]。
- 拓扑同构利用**: 支路 Hessian、Jacobian 与 Ybus 结构同构, 可以直接用 Ybus 的 nnz 索引寻址, 无需额外查找表。

5. **暖启动**：先做 NR 潮流得到 V_a/V_m 初值，OPF 典型 14 迭代收敛（无暖启动约 20-30 迭代）。

文档由代码自动分析生成，对应 `commit 02985b5`（`opf_exp` 分支）。